Universidad Nacional Autónoma de México

Facultad de Ingeniería

Ejemplo Flyweight: Generador de Árboles

Programación Orientada a Objetos

Profesor: Edgar Tista García

Integrantes

Robles Cancino Arturo Andres
Rodríguez Cárcamo Ditter Emmanuel
Santamaría Viveros César Yurem

2 de Noviembre, 2026

1 Diseño del ejemplo: bosque con el patrón Flyweight

En este ejemplo se modela un bosque que contiene distintos árboles. Cada árbol tiene características propias, como su tipo, color y altura. Sin embargo, es posible que muchos árboles compartan exactamente las mismas características, por lo que no es necesario crear un objeto independiente para cada uno.

Para resolver este problema se utiliza el patrón de diseño Flyweight, que permite compartir objetos que contienen información común y mantener por separado los datos que cambian dependiendo de cada árbol.

2 Cómo funciona el código

2.1 El Flyweight: `Arbol`

La clase `Arbol` representa el objeto que puede compartirse entre varios árboles del bosque. Esta clase almacena las características comunes entre los árboles, como el tipo, el color y la altura.

Estas características forman parte del estado intrínseco, debido a que se mantienen iguales para todos los árboles que comparten esa instancia.

Además, la clase cuenta con el método `dibujar(int x, int y)`, que recibe las coordenadas del árbol como parámetros. De esta manera, la posición no necesita almacenarse dentro del objeto compartido.

2.2 La fábrica: `ArbolFactory`

La clase `ArbolFactory` se encarga de administrar y reutilizar los objetos de tipo `Arbol`.

Para conseguirlo, utiliza un `HashMap`, donde almacena las instancias creadas de acuerdo con sus características: tipo, color y altura.

Cuando se solicita un árbol, la fábrica primero verifica si ya existe un objeto con esas características. Si existe, devuelve la instancia almacenada; de lo contrario, crea un nuevo objeto y lo guarda para poder reutilizarlo posteriormente.

Esto evita crear múltiples objetos con información idéntica y permite que varios árboles compartan una misma instancia.

2.3 El cliente: `Main`

La clase principal representa al cliente que utiliza la fábrica para obtener los árboles.

En lugar de crear directamente objetos de tipo `Arbol`, solicita las instancias mediante `ArbolFactory`. Posteriormente, utiliza el método `dibujar()` enviando las coordenadas correspondientes a cada árbol.

De esta forma, varios árboles pueden compartir el mismo objeto, pero conservar posiciones diferentes dentro del bosque.

2.4 Ejemplo de ejecución

En la salida se puede observar el tipo, color y altura de cada árbol, así como su posición dentro del bosque y se puede ver que habiendo árboles con las mismas características, se reutiliza la misma instancia de `Arbol`.

La siguiente captura muestra un ejemplo de la ejecución del programa:

```
Creando nuevo tipo de arbol: Pino
Reutilizando arbol: Pino
Creando nuevo tipo de arbol: Roble
Reutilizando arbol: Roble
Reutilizando arbol: Pino
Arbol: Pino | Color: Verde | Altura: 45 | Posicion: (10,20)
Arbol: Pino | Color: Verde | Altura: 45 | Posicion: (30,40)
Arbol: Roble | Color: Cafe | Altura: 30 | Posicion: (50,60)
Arbol: Roble | Color: Cafe | Altura: 30 | Posicion: (70,80)
Arbol: Pino | Color: Verde | Altura: 45 | Posicion: (90,100)
```

3 Estado intrínseco y extrínseco

En este ejemplo se identifican dos tipos de estado:

- **Estado intrínseco:** corresponde al tipo, color y altura del árbol. Esta información se almacena dentro de la clase `Arbol` y puede compartirse entre diferentes árboles.
- **Estado extrínseco:** corresponde a las coordenadas (x, y) de cada árbol. Esta información depende de la ubicación particular de cada árbol y se proporciona al método `dibujar()` cuando se necesita.

La separación de estos estados permite reutilizar los datos que no cambian y mantener de manera independiente la información que depende de la ubicación de cada árbol dentro del bosque.

4 ¿Por qué es adecuado aplicar Flyweight aquí?

Por ejemplo, si existen cien árboles de tipo pino, con el mismo color y altura, no es necesario almacenar cien objetos con exactamente la misma información. La fábrica puede crear una sola instancia de `Arbol` y devolver esa misma referencia cada vez que se solicite un árbol con dichas características.

Las principales razones por las que este código aplica el patrón son:

1. **Reutilización de objetos:** la fábrica permite compartir instancias que tienen las mismas características.
2. **Reducción del consumo de memoria:** se evita crear objetos duplicados que almacenan información idéntica.
3. **Separación de estados:** las características comunes se almacenan en `Arbol`, mientras que las coordenadas se proporcionan externamente.
4. **Uso de una fábrica:** la clase `ArbolFactory` centraliza la creación y recuperación de los objetos compartidos.
5. **Independencia de la posición:** un mismo objeto puede utilizarse para representar árboles ubicados en diferentes coordenadas.

Conclusión

Se considera que este ejemplo implementa el patrón de diseño Flyweight porque permite reducir la cantidad de objetos creados mediante la reutilización de instancias que comparten las mismas características.

La clase `Arbol` almacena el estado intrínseco, compuesto por el tipo, color y altura, mientras que las coordenadas representan el estado extrínseco. Por otra parte, `ArbolFactory` administra las instancias para evitar crear objetos repetidos.

De esta manera, el programa demuestra cómo el patrón Flyweight permite optimizar el uso de memoria cuando se necesita representar una gran cantidad de elementos que comparten información.